

Dynamic Storyboard Generation in an Engine-based Virtual Environment for Video Production

Anyi Rao*
Stanford University
USA
anyirao@stanford.edu

Xuekun Jiang*
Shanghai AI Lab
China
jiangxuekun@pjlab.org.cn

Yuwei Guo
Shanghai AI Lab
China
guoyuwei@pjlab.org.cn

Linning Xu
CUHK
Hong Kong, China
linningxu@ie.cuhk.edu.hk

Lei Yang
Shanghai AI Lab
China
yanglei@pjlab.org.cn

Libiao Jin
CUC
China
libiao@cuc.edu.cn

Dahua Lin
Shanghai AI Lab, CUHK
Hong Kong, China
dhlin@ie.cuhk.edu.hk

Bo Dai
Shanghai AI Lab
China
daibo@pjlab.org.cn

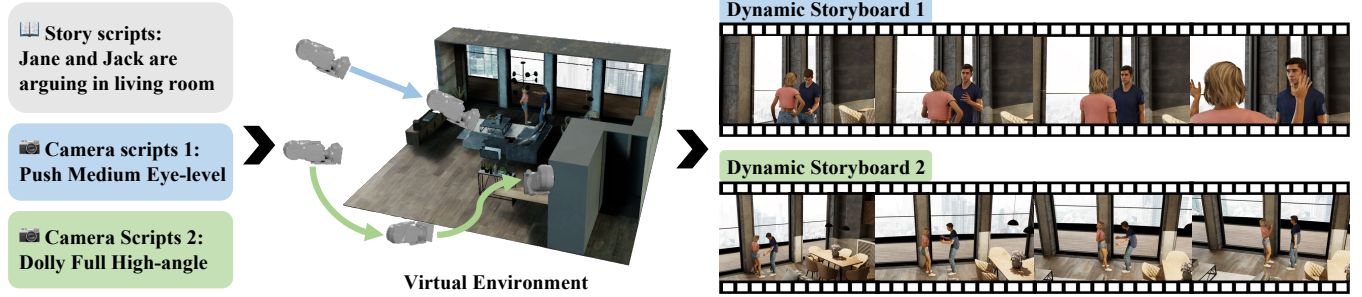


Figure 1: We present Virtual Dynamic Storyboard (VDS) that takes user input *story* and *camera* scripts and automatically composes dynamic storyboards in an engine-based virtual environment for pre-visualization. Here we show two results produced by VDS with top-ranked scores of quality. Video demos can be found in the supplementary.

CCS CONCEPTS

• **Information systems** → Multimedia content creation.

KEYWORDS

virtual storyboard, cinematography, neural networks

ACM Reference Format:

Anyi Rao, Xuekun Jiang, Yuwei Guo, Linning Xu, Lei Yang, Libiao Jin, Dahua Lin, and Bo Dai. 2023. Dynamic Storyboard Generation in an Engine-based Virtual Environment for Video Production. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Posters (SIGGRAPH '23 Posters)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3588028.3603647>

1 INTRODUCTION

Born in 1930, storyboarding technique helps video creators design each shot, figure out potential problems, and communicate ideas to save time and resources raised in practical video production [Wikipedia 2023]. A conventional storyboard represents each shot with one still frame, depicting the scene layout, character actions, and camera parameters such as scale and angle. However, such a static storyboard is often dry and rigid, since it lacks the

intrinsic capability to faithfully demonstrate dynamic semantics including character and camera movements. Moreover, most conventional storyboards are hand-painted sketches that require both time and drawing skills. A rudimentary storyboard of a target video lasting for minutes can take artists hours to finish, let alone amateur creators who lack skills.

In this paper, we propose a novel virtual dynamic storyboard approach in an efficient and customized way for amateurs. It follows a “propose-simulate-discriminate” paradigm. For each shot, it first translates the inputted high-level *story* and *camera* script in the form of $((\langle \text{char} \rangle \langle \text{do} \rangle \langle \text{sth/swh} \rangle))$ tuples for story scripts and $((\langle \text{movement} \rangle \langle \text{scale} \rangle \langle \text{angle} \rangle))$ tuples for camera scripts, into proposals of shot hyperparameters. Each proposal is then executed by the simulation module built on top of a modern graphic engine (e.g., Nvidia [2023]; Unity [2023]; Unreal [2023]) to acquire its corresponding rendered output. Finally, a data-driven discriminator is adopted to rank all these rendered outputs as a recommendation for users to choose the final dynamic demonstration of this shot.

Our key insights are as follows: 1) The enormous action space for all possible character and camera trajectories on a frame-by-frame basis can easily lead to low-quality shot candidates in terms of inter-frame abruptness and less meaningful camera motions. Our shot-based story and camera proposal generation significantly reduces the search space for appropriate camera configurations, further ensuring the results are more plausible. 2) By wrapping the graphic engine into a simulation module, the proposed approach significantly improves its quality and efficiency, thanks to engines’ highly-optimized computation pipeline and high rendering quality.

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH '23 Posters, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0152-8/23/08.

<https://doi.org/10.1145/3588028.3603647>

3) There might be several plausible shots satisfying the requirements of story and camera scripts and no universal criteria can serve as direct supervision for generated shots. We resort to a data-driven learning approach that aims to automatically discover rules from professional manual-created clips.

2 SYSTEM OVERVIEW

Our Virtual Dynamic Storyboard (VDS) first proposes several candidates of executable parameters for each shot’s scripts (Sec. 2.1 and Sec. 2.2), following the story and cinematic rules. The proposal storyboard videos are then rendered by an engine-based simulator that satisfy the above parameters. Through class-aware contrastive learning, a shot ranking discriminator helps to rank the generated proposals and assists users in selecting their favorites (Sec. 2.3).

2.1 Story Script Proposal

One of the raw inputs to VDS is a sentence of story scripts in the format of $\langle \langle \text{char} \rangle \langle \text{do} \rangle \langle \text{sth/sw} \rangle \rangle$. Although graphic engines have strong ability in scene establishment and character animation with manual force (mouse/keyboard clicking/typing), these functions cannot be directly called with the aforementioned story scripts. To fulfill this goal, we develop an *engine-based simulation module* with a series of automatic APIs including: scene selection, character placement and animation. The API will translate each shot’s script into story parameters s in *subspace* S , which can be executed for character animation in the storyboard,

$$s = (\mathbf{a}_n, \mathbf{p}_{n,m}), \quad s \in S. \quad (1)$$

where $\mathbf{a}_n$ is a corresponding pre-recorded proposal animation clip and $\mathbf{p}_{n,m}$ represent a motion trajectory for $\mathbf{a}_n$.

2.2 Camera Script Proposal

To associate the produced shots with the classical cinematic style filming rules, and provide easy control to users, we introduce three dimensions for shot type control and allow the input in the format of $\langle \langle \text{movement} \rangle \langle \text{scale} \rangle \langle \text{angle} \rangle \rangle$, which are widely used in the filming industry to increase production efficiency [Rao et al. 2020].

To fulfill the control on it, we define our camera trajectory as a list of 7DoF camera parameters, while other camera parameters e.g. ISO, shutter speed are set as default:

$$\mathbf{c} = \{(x^t, y^t, z^t), (\alpha^t, \beta^t, \gamma^t), f^t\}, \quad t \in [0, T], \quad \mathbf{c} \in C, \quad (2)$$

where (x^t, y^t, z^t) stands for the position in world coordinates. The roll α^t , pitch β^t , and yaw γ^t describe the camera rotations along the Z_C, X_C, Y_C axes in the camera local coordinate, where the upward direction is specified by a “Look-At” constraint. Instead of freely searching from the general camera action space, we identify a set of camera *subspaces* C that defines meaningful camera trajectories to support eleven movement types, eight scale types, and three angle types. The details are specified in the supplementary.

2.3 Shot Ranking Discriminator

For a given story and camera script, the subspaces defined above produce $S \cdot C$ plausible shot proposals rendered by our simulator differing in their specific parameters on paths and views.

We train a binary classifier with the following loss using the professional manual-created clips as positive samples and the randomly generated samples as negative ones, under the assumption that the former represents higher quality.

Table 1: Pair-wise comparison of hand-paint and VDS on time cost per shot in minutes and binary voting.

Designer Type	Approach	Time Cost per shot	Reach Expectation	Camera Flexibility	Story Flexibility
Beginner	Hand-paint VDS	6.69 min	16.67%	8.33%	75.00%
		3.42 min	83.33%	91.67%	25.00%
Skilled user	Hand-paint VDS	12.73 min	25.00%	12.50%	62.50%
		3.13 min	75.00%	87.50%	37.50%

$$\mathcal{L}_b = -y \log(p_b) + (1 - y) \log(1 - p_b). \quad (3)$$

During inference, the generated samples can be sorted by their classification scores for being categorized as “professional”. And our goal is to find the one with the highest score that fools the network into treating them as professional samples.

Considering the variance among generated samples caused by their intrinsic shot styles, we add a **class-aware** loss to encourage it to be more class type specific and be expert in determining the corresponding shot type’s quality:

$$\mathcal{L}_c = - \sum_{c=1}^M y \log(p_c), \quad (4)$$

where M is the total number of shot types in training data.

To better select the high-quality generated shots, we need to magnify the feature difference among samples of different quality. We propose to include a **contrastive** objective, to learn discriminative features among different samples for each shot, which is helpful to determine the generated quality,

$$\mathcal{L}_q = - \log \frac{\exp(z_q z_+ / \tau)}{\sum_{k=1}^K \exp(z_q z_k / \tau)}. \quad (5)$$

The loss is computed over one sample z_q and K other samples $\{z_k\}$ in the training set. z_+ comes from different frames at other timestamps within the same shot as the target sample z_q .

3 EXPERIMENTS

We also design a two-stage user interface to help user to generate storyboard, as shown in the demo video. To evaluate the performance of VDS in practical usage, we invite 20 amateur designers to compare the usage of hand-paint and our VDS. 8 of them are **skilled users** who learn painting over two years while the rest 12 of them are **beginners** without painting skills. The results in Tab. 1 show that our tool can effectively compose dynamic storyboards and assist amateurs in their creation.

4 CONCLUSION

Virtual Dynamic Storyboard (VDS) is a semi-auto storyboard creation tool performing on virtual environments. It takes user-specified story script and camera script as inputs and proposes multiple plausible character and camera trajectories that can be rendered to be dynamic shots. A shot ranking discriminator then sorts these generated shots with class-aware contrastive objectives to help users select their favorites.

REFERENCES

- Nvidia. 2023. Omniverse Platform. <https://www.nvidia.com/omniverse/>.
- Anyi Rao, Jiaze Wang, Linning Xu, Xuekun Jiang, Qingqiu Huang, Bolei Zhou, and Dahua Lin. 2020. A Unified Framework for Shot Type Classification Based on Subject Centric Lens. In *The European Conference on Computer Vision (ECCV)*.
- Unity. 2023. Real-Time Development Platform. <https://unity.com/>.
- Unreal. 2023. Real-Time 3D Creation Tool. <https://www.unrealengine.com/>.
- Wikipedia. 2023. Storyboard. <https://en.wikipedia.org/wiki/Storyboard>.